Web3 支付：Escrow 托管 + 钱包签名 + 链上监听

gomall 把” 第三方支付” 换成” 可审计代码” 的工程链路

gomall · 业务侧 deck

今日大纲

业务目的：传统支付吃不下的那批订单

Escrow 合约：把” 第三方” 换成代码

钱包签名：EIP-191 personal_sign + nonce 防重放

链上对账：链是异步的

业务边界：业务码 / 角色 / 合规

业务目的：传统支付吃不下的那批订单

为什么 gomall 引入 Web3 支付

- **加密原生用户**：DeFi 用户钱包里全是链上资产，刷卡通道用不上。
- **跨境结算**：传统通道 1–3% 手续费 + T+1 到账，跨境争议解决依赖银行 SWIFT。
- **资金托管增信**：买卖双方互不信任的大额订单，把”中介”从银行换成审计过的合约。
- **业务收益**：开新渠道，不是替代旧通道，跑在传统 `/paydown` 旁边。

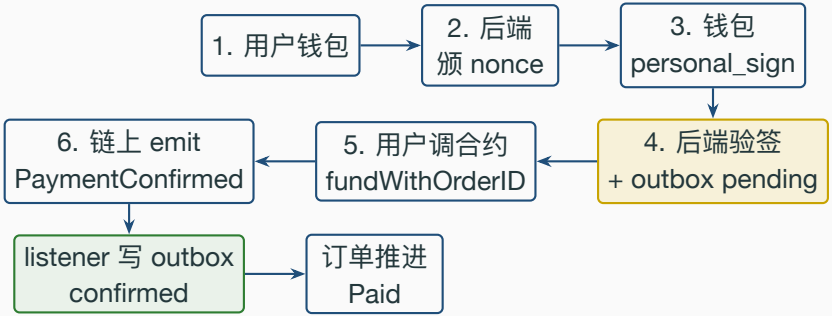
routes/router.go:113–127 paydown（fiat）与 paydown/crypto（web3）平行挂载

传统第三方支付 vs Escrow 托管

维度	第三方支付	Web3 escrow
信任锚	银行 / 支付牌照	智能合约 + 链
结算时间	T+0 / T+1	1 块 30–60s, 6 块 ≈ 90s
跨境费用	1–3%	gas 固定 + 兑换损耗
退款主导	平台仲裁	合约 + 仲裁人
身份证明	实名 + 卡密	钱包私钥 EIP-191 签名

Web3 不替代第三方支付，而是**补齐**传统通道吃不下的订单。*contracts/README.md:1–36* 合约职责与状态机不变式

端到端 6 步链路

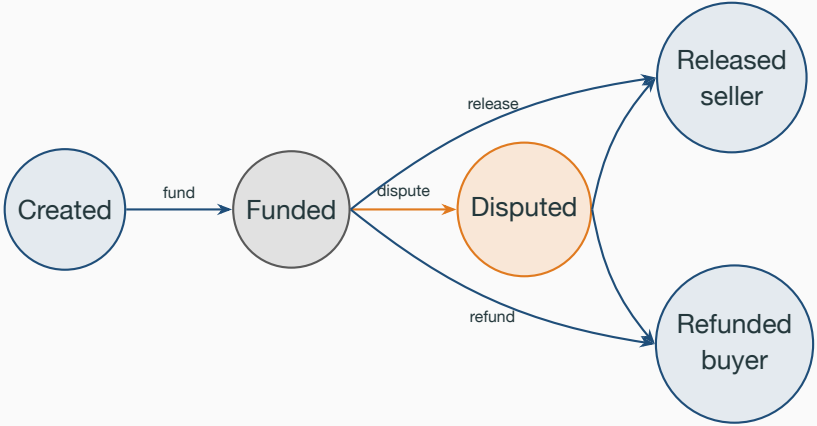


钱进合约前订单永远 pending；钱进合约后由 listener 兜底推进，避免”先付款后失踪”。*routes/router.go:123–127*

GET paydown/crypto/nonce + POST paydown/crypto

Escrow 合约：把” 第三方” 换成代码

Escrow 状态机：5 个状态、3 个角色



不变式：**Created** → **Funded** 只能发生一次（`inState(Created)` 守卫）；**Funded / Disputed** → **Released | Refunded** 终态不可逆；所有 immutable 参数（buyer/seller/arbiter/amount）部署后无法改。*contracts/Escrow.sol:13–19 enum；:68–82 fund/fundWithOrderID；:85–107 release/refund/dispute*

合约状态 ↔ 订单状态映射

合约状态	订单状态	触发动作
Created	已下单未支付	合约部署完成
Funded	已支付待发货	买家 `fundWithOrderID`
(链下)	商家发货 / 物流	链下流程，不影响合约
Released	已完成	买家 / 仲裁人 `release`
Refunded	已退款	卖家 / 仲裁人 `refund`
Disputed	售后争议	买家或卖家 `dispute`

链上只记” 钱在谁那里”；订单

详情 / 物流 / 对账都在链下。

合约的不变式 = 业务对外的最低承诺。*contracts/README.md:31–36* 合约不变式条款

Escrow.sol 关键路径：fundWithOrderID

Listing 1: *contracts/Escrow.sol:74–82 fundWithOrderID*

```
74 function fundWithOrderID(bytes32 _orderID)
75    external payable onlyBuyer inState(State.Created)
76 {
77    if (msg.value != amount) revert WrongAmount(amount, msg.value);
78    orderID = _orderID;
79    state = State.Funded;
80    emit Funded(msg.value);
81    emit PaymentConfirmed(_orderID, msg.sender, msg.value);
82 }
```

逐行解读：

- `onlyBuyer + inState(Created)`：只能 buyer 调，且只能在 Created 态调，状态机入口唯一。
- `msg.value != amount`：链上强校验金额，少 1 wei 也 revert，业务侧零信任。
- `orderID = _orderID`：链下订单号写进 storage，listener 拿它 join 业务库。

钱包签名：EIP-191 personal_sign + nonce 防重放

为什么不能把私钥交给后端

- 私钥 = 钱包的全部，给了后端就不再是”用户的钱包”。
- 业界做法：后端给一段消息 → 钱包客户端内签 → 后端 `ecrecover` 反推地址。
- 业务收益：
  - **不可否认**：地址匹配 = 用户授权过；
  - **零私钥泄漏面**：签名发生在钱包客户端内部；
  - **防钓鱼**：消息明文人类可读，钱包弹窗能让用户看清自己在签什么。

pkg/web3/signature/verify.go:1–13 包注释设计意图

签名消息模板：chainID + orderID + nonce 全钉进去

`gomall:paydown:order={id}:nonce={n}:chain={cid}`

↓

EIP-191: keccak256(”\x19Ethereum Signed Message:\n<len>” + msg)

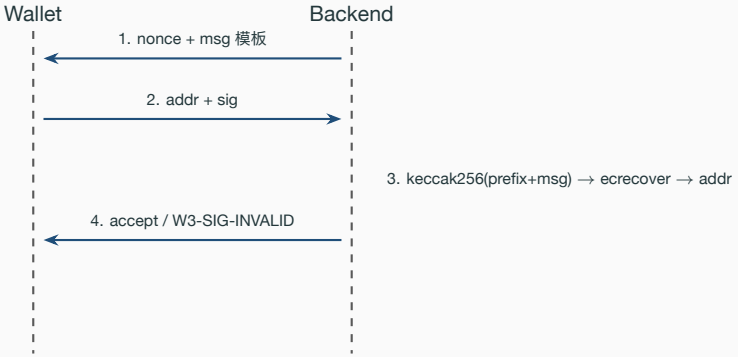
↓

ecrecover(hash, sig) → address　vs　钱包地址

- **chainID 入消息**：防一条签名在 mainnet / L2 之间互通造成跨链重放。
- **orderID + nonce**：绑定订单 + 防同链重放。
- **前缀 `gomall:paydown:`**：让用户在钱包弹窗里识别” 我在给 gomall 付款”。
- **v 值归一**：钱包 27/28，go-ethereum 还原 pubkey 要 0/1，必须归一。

service/payment_crypto.go:39–46 signMessageTemplate pkg/web3/signature/verify.go:31–67 VerifyPersonalSign

验签时序：4 拍完成” 是不是这个钱包”



全过程后端不接触私钥；

签名失败 → `W3-SIG-INVALID`；nonce 失效 → `W3-NONCE-EXPIRED`。

service/payment_crypto.go:104–124 VerifyAndPark 第 1–2 段

VerifyPersonalSign：EIP-191 + ecrecover 12 行

Listing 2: *pkg/web3/signature/verify.go:31–67*

```go
func VerifyPersonalSign(addr string, msg []byte, sig []byte) (bool, error) {
    if len(sig) != personalSigLen { return false, fmt.Errorf("...") }
    if !common.IsHexAddress(addr) { return false, errors.New("...") }
    rsv := make([]byte, personalSigLen)
    copy(rsv, sig)
    switch rsv[64] {
    case 27, 28: rsv[64] -= 27
    case 0, 1: // already normalized
    default: return false, fmt.Errorf("非法 v 值: %d", rsv[64])
    }
    hash := personalSignHash(msg) // EIP-191 prefix + keccak256
    pubKey, err := crypto.SigToPub(hash, rsv) // ecrecover
    if err != nil { return false, err }
    recovered := crypto.PubkeyToAddress(*pubKey)
    expected := common.HexToAddress(addr)
    return bytes.Equal(recovered.Bytes(), expected.Bytes()), nil
}
```

IssueNonce + VerifyAndPark 主流程拼图

Redis key `web3:nonce:{userID}:{orderID}`（用户 + 订单双维度）TTL 5min；二次签发自动覆盖旧 nonce；消费走 Lua `GET+DEL`（server 端原子，并发只能一个拿到 1）；Pending 占位 `paydown:web3:pending:{orderID}` TTL 30min。*repository/cache/web3.go:21–26 TTL；:60–89 consumeNonceScript*

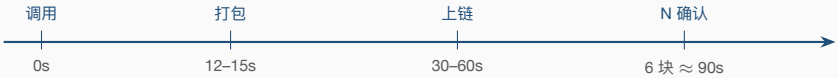
Listing 3: *service/payment_crypto.go:49–148* 节选

```go
// IssueNonce: 校验订单归属 + 状态 + 颁发
order, _ := dao.NewOrderDao(ctx).GetOrderById(req.OrderId, u.Id)
if order.Type != consts.UnPaid { return errors.New("订单状态非未支付") }
nonce, _ := randomNonce() // 16 字节 -> 32 hex
cache.PutWeb3Nonce(ctx, u.Id, req.OrderId, nonce) // SET + 5min TTL

// VerifyAndPark: 消费 nonce -> 验签 -> 事务内写 outbox
if err := cache.ConsumeWeb3Nonce(ctx, u.Id, req.OrderID, req.Nonce); err != nil {
    return err // -1 missing / -2 mismatch -> W3-NONCE-EXPIRED
}
msg := []byte(BuildSignMessage(req.OrderID, req.Nonce, req.ChainID))
ok, _ := web3sig.VerifyPersonalSign(req.WalletAddr, msg, sigBytes)
if !ok { return errors.New("签名与钱包地址不匹配") } // W3-SIG-INVALID
```

链上对账：链是异步的

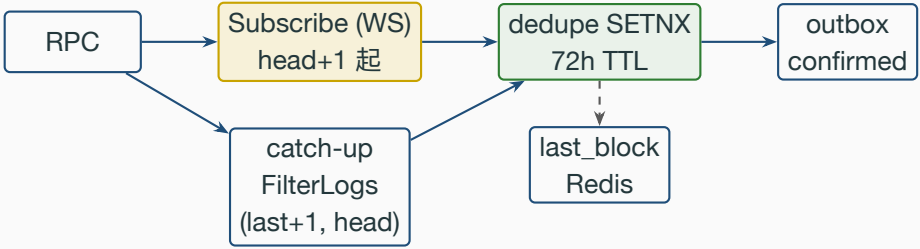
链是异步的：4 个时间锚点



- **0s 调用**：钱包广播 tx 到内存池。
- **12–15s 打包**：validator 把 tx 装进区块。
- **30–60s 1 确认**：业务可显示”已上链”。
- **90s 6 确认**：抗 reorg，金额越大 N 越大（大额 12–20 块）。

链下到链上不是同步调用；用户体验走 ticket 模式：返回 `W3-PENDING` + 客服话术”等待区块确认（约 30–60s）”。*service/web3/listener.go:177–230 connectAndServe head* 推进

listener 的 3 项基本功：重连 / catch-up / 幂等



- 启动 / 重连后先 `catchUp(last+1, head)` 把停机漏掉的事件批量回放。
- 每条 log 落 outbox 前过 `web3:event:{txhash}:{logIdx}` SETNX 72h 幂等。
- 处理一条就 `saveLastBlock`，重启不重做已成功事件。
- 重连退避：**1s** → **2s** → **4s** → **…**→ **60s 封顶**，连上后 reset。

service/web3/listener.go:232–248 catchUp；:296–301 dedupeKey；:316–326 tryClaim

service/web3/listener.go:25–31 eventDedupeTTL=72h；:364–377 backoff

StartPaymentListener 主循环 + connectAndServe

Listing 4: *service/web3/listener.go:177–230 connectAndServe*

```go
client, _ := l.dial(ctx, l.rpcURL)
defer client.Close()
head, _ := client.BlockNumber(ctx)
last := l.loadLastBlock(ctx)
if last == 0 || last > head { last = head }
if last < head {
   l.catchUp(ctx, client, last+1, head) // 回放停机期间漏掉的事件
}
l.saveLastBlock(ctx, head)
logsCh := make(chan types.Log, subscribeBuffer) // = 64
sub, _ := client.SubscribeFilterLogs(ctx,
   l.query(big.NewInt(int64(head+1)), nil), logsCh)
defer sub.Unsubscribe()
l.backoff.reset() // 连上立即清退避
for {
   select {
   case <-ctx.Done(): return ctx.Err()
   case err := <-sub.Err(): return err // -> 外层重连
   case lg := <-logsCh:
```

容错矩阵：reorg / 重启 / 重复 / RPC 断

异常	listener 行为	业务影响
RPC 断连	backoff 1s → 60s 指数重连	短暂延迟，不掉单
进程重启	loadLastBlock 后 catchUp(last+1, head)	不掉单
log.Removed	reorg 撤销，跳过	依赖 SETNX 兜底
重复事件	SETNX 72h key 已存在 return	不重复推进订单
RPC 慢	Subscribe 不阻塞主流程，buffer=64	log 缓冲，不丢

复用

deck 04 outbox 总线：listener 写入 `web3.payment.confirmed`，与传统支付的 `order.paid` 走同一条总线，对账可复用。*service/web3/listener.go:155–175 run()* 退避重连；*:218–221 reorg* 跳过 *service/web3/listener.go:296–301 dedupeKey 72h*；*:303–311 outbox.Insert*

链下 vs 链上时间线对比 + ticket UX

Web3 不可能”秒级到账”。业务侧 UX 走 ticket 模式：钱包签名后立即返回 `W3-PENDING` + ticket#，前端轮询订单状态；1 确认前不发货，6 确认（大额 12+）才走 release；客服话术 `W3-PENDING` = ”等待区块确认（约 30–60s）”，不是失败。

业务边界：业务码 / 角色 / 合规

Web3 业务码 + 客服话术

业务码	含义	客服话术
W3-NONCE-EXPIRED	nonce 5min 过期或被消费	” 请重新发起支付”
W3-SIG-INVALID	签名地址与提交不匹配	” 请用绑定钱包重试”
W3-PENDING	链上未确认	” 等待区块确认（约 30–60s）”
W3-REORG	链上 reorg 撤销	” 网络回滚，已自动退款”

业务码不暴露技术细节，但要给客服**足够区分**的话术。

`W3-PENDING` 与 `W3-NONCE-EXPIRED` 表面像，应对策略完全不同：前者等链确认，后者重发 nonce。*repository/cache/web3.go:29–33 ErrWeb3NonceMissing / ErrWeb3NonceMismatch*

各角色关心什么

角色	关心	看哪里
用户	钱安全吗	合约审计报告 + 多签 + 退款流程
商家	何时到账	链上确认数 + release 时机
客服	链上失败咋办	W3-* 业务码对应处理 SOP
SRE	节点稳定否	RPC 多源切换 + listener 健康
法务	合规边界	KYC + 多签 + 司法管辖
财务	链上链下并存对账	outbox + 链上事件 + 法币入账

`outbox` 总线让财务对账

拿到**统一格式**的事件：channel = fiat / web3-eth / web3-usdc，按 `order_id` 维度 join。**业务边界**：链 / 币 / gas 三选一组合，MVP 推荐 **L2 + 稳定币 + 用户自付 gas**；正式运营后逐步上 gas 补贴 + 多链路由。*contracts/README.md:88–103* 多签建议；*service/payment_crypto.go:41* 多 *chainID* 接口

风险与合规 + Arbiter 多签升级路线图

- **合约 immutable**：bug 只能换合约；缓解：第三方审计 + 充分单测。
- **私钥丢失**：链上无” 重置密码”；缓解：仲裁人核实身份后 refund 到新地址。
- **gas 飙升**：mainnet 拥堵 tx 失败；缓解：默认 L2，fallback mainnet。
- **Reorg**：浅确认下 release 可能撤销；缓解：大额提高 N 确认 + outbox 兜底。
- **AML / OFAC**：制裁名单地址必须拒绝；缓解：验签阶段地址过滤。

阶段	arbiter 形态	适用金额上限
MVP	平台 EOA	等值 $100 以内
2	2-of-3 多签	等值 $10,000 以内（Gnosis Safe）
3	DAO 治理	超大额 / 跨组织

service/web3/listener.go:218–221 reorg 跳过；contracts/README.md:88–103 arbiter 多签

从技术决策回看业务依据 + 最终目标

技术选择	业务依据
EIP-191 personal_sign	钱包弹窗让用户看到明文，钓鱼可识别
nonce 走 Redis + Lua	防重放门槛要原子，单实例语义不够
chainID 入签名消息	防 mainnet / L2 跨链重放
合约 emit PaymentConfirmed	listener 订一个 event 就能精确对账
listener 写 outbox 而非直推订单	复用 deck 04 的最终一致体系
SETNX 72h 事件级幂等	reorg / 重连 / 重启不会重复推进
arbiter 走多签路线	单签平台跑路风险无法对外解释

业务价值不在”潮”，而在于**把”第三方”从”中介机构”换成”可审计的代码”**。

routes/router.go:123–127 Web3 路由接入位

面试 Q&A（上）

Q1：链 reorg 把已 release 的资金撤销了怎么办?

A：金额越大 N 确认越多（大额至少 12–20 块）；listener 监听 `lg.Removed` 跳过 reorg log，订单状态不会被错误推进；已发货走对账兜底 + 仲裁人介入。

Q2：用户钱包私钥丢了订单怎么处理?

A：链上没有” 重置密码”。资金在 Funded 态可由仲裁人核实身份后 refund 到新地址；已 Released 给商家则只能走链下补偿。

Q3：gas 飙升导致用户支付 tx 失败怎么补救?

A：默认路由到 L2；nonce 5min 内允许用户重新提交带更高 gas 的 tx；超时则重新颁发 nonce + 签名。

面试 Q&A（下）

Q4：和传统第三方支付的对账如何并存?

A：两条通道都把支付事件落到同一条 outbox 总线，事件加 channel 字段（fiat / web3-eth / web3-usdc）。对账任务按 `order_id` join 法币入账 + 链上 confirmed event。

Q5：为什么 nonce 必须后端发，前端自己生成不行吗?

A：前端可生成的 nonce 后端没法保证唯一性也没法做时效控制；攻击者拿到一次有效签名后可无限重放。后端 Redis + Lua 原子 GET+DEL 才能保证” 一次性”。

Q6：listener 进程重启会不会丢事件?

A：不会。`loadLastBlock` 从 Redis 拿到最后处理过的块号，`catchUp(last+1, head)` 用 `FilterLogs` 批量回放停机期漏掉的事件，再叠加 SETNX 72h 幂等，重启不会重做也不会丢做。

代码位置一览

Escrow 合约 + 状态机 *contracts/Escrow.sol:7–113*

钱包验签 EIP-191 *pkg/web3/signature/verify.go:31–82*

Nonce 颁发 + 验签业务 *service/payment_crypto.go:49–160*

Web3 Redis 协议（nonce/pending） *repository/cache/web3.go:21–101*

链上 listener *service/web3/listener.go:108–326*

退避重连 *service/web3/listener.go:360–377 backoff 1s→60s*

路由接入位 *routes/router.go:123–127*

合约部署 / 多签建议 *contracts/README.md:1–104*

现状：合约 + binding 在 feat/web3-escrow PR；**未上链**，上线前需第三方审计。